

Reviewer Pack — Minimal Rank of Tensor Product of Polynomial Algebras Bounds...

Entry 275c21ac1aad · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 04:33:18 UTC

Minimal Rank of Tensor Product of Polynomial Algebras Bounds Frege Proof Depth

Entry ID: 275c21ac1aad **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 04:33:18 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Tensor Products of Polynomial Algebras)

Field B (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

[‘For any given n -variables CNF F , let A_F be the algebra generated by F , and B_F the algebra generated by all possible assignments of F . The conjecture states that the minimal rank of the tensor product of algebras A_F and B_F is upper bounded by $2^n - \log_2(n)$.’, ‘Moreover, for any CNF F with m clauses and n variables, if there exists a Frege proof of F with depth d , then the minimal rank of the tensor product of algebras A_F and B_F is greater than or equal to $2^{(n-d)} - \log_2(2^{(n-d)})$.’, ‘This implies that for small values of n , high Frege proof depths correspond to high ranks in the tensor product.’]

Rationale (proposer’s reasoning):

[‘The connection between algebraic structures and complexity theory is well-established. Tensor products of polynomial algebras have been used to study various problems in computational geometry and algebraic geometry. This conjecture aims to leverage this connection to provide insights into Frege proof complexity.’, ‘By studying the rank of the tensor product, we can potentially uncover new relationships between the structure of the CNF and its proof complexity.’, ‘The proposed mapping is computationally feasible since it involves straightforward operations on polynomials.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8af92ca471b9e3b0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman’s rank correlation coefficient between the minimal rank of the tensor product and the Frege proof depth exceeds 0.7 for all n in $\{5, 10, 20, 40\}$, across at least 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tensor product polynomial algebras minimal rank AND Frege proof complexity - algebraic combinatorics tensor products polynomial algebras relation Frege proof depth - Frege proof depth lower bound on tensor product algebraic structure

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows = len(matrix)
    cols = len(matrix[0])
    rref = [[Fraction(0) for _ in range(cols)] for _ in range(rows)]

    for i in range(min(rows, cols)):
        max_row = i
        for j in range(i+1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j

        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        pivot = matrix[i][i]
        if pivot == 0:
            continue

        for j in range(cols):
            matrix[i][j] /= pivot

        for j in range(rows):
            if j != i:
                factor = matrix[j][i]
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]
```

```

rank = 0
for row in rref:
    if any(row):
        rank += 1

return rank

def tensor_product(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    result = [[Fraction(0) for _ in range(cols_A * cols_B)] for _ in range(rows_A * rows_B)]

    for i in range(rows_A):
        for j in range(cols_A):
            for k in range(rows_B):
                for l in range(cols_B):
                    result[i*rows_B + k][j*cols_B + l] = A[i][j] * B[k][l]

    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 20, 40]
    results = []

    for n in n_values:
        m = random.randint(1, 2*n)
        CNF = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]

        A_F = [sum(CNF[i][j] * x[j] for j in range(n)) for i in range(m)]
        B_F = [sum(CNF[i][j] * x[j] for j in range(n)) for i in range(m) for x in itertools.product(
            [-1, 1], repeat=n)]

        tensor_product_matrix = tensor_product(A_F, B_F)
        rank_value = gaussian_elimination(tensor_product_matrix)

        # Placeholder for Frege proof depth calculation
        frege_depth = random.randint(1, n)

        results.append({
            "n": n,
            "m": m,
            "rank_value": rank_value,
            "frege_depth": frege_depth
        })

    correlation_coefficient = 0.0
    for result in results:
        correlation_coefficient += (result["rank_value"] - mean_rank) * (result["frege_depth"] - mean_frege_depth)
    correlation_coefficient /= len(results)

    return {
        "metric_name": "Spearman's rank correlation coefficient",

```

```

        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "conjecture_holds": abs(correlation_coefficient) > 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 31)]

    mean_rank = 0.0
    mean_frege_depth = 0.0
    conjecture_holds_count = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        mean_rank += trial_result["metric_value"]
        mean_frege_depth += abs(trial_result["rank_value"] - trial_result["frege_depth"])
        if trial_result["conjecture_holds"]:
            conjecture_holds_count += 1

    mean_rank /= len(seeds)
    mean_frege_depth /= len(seeds)
    support_fraction = conjecture_holds_count / len(seeds)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={mean_frege_depth} support_fraction={support_fraction}")
    elif support_fraction > 0:
        print(f"RESULT: FALSIFIED counterexample=\"not enough evidence\" first_failing_seed=1")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3863d75d.py", line 115, in <module>
 trial_result = run_trial(seed)
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_3863d75d.py", line 77, in run\_trial  
 A\_F = [sum(CNF[i][j] \* x[j] for j in range(n)) for i in range(m)]  
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3863d75d.py", line 77, in <genexpr>
 A_F = [sum(CNF[i][j] * x[j] for j in range(n)) for i in range(m)]
 ~~~~~  
 ^

NameError: name 'x' is not defined

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 12

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11398        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11023        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12275        |
| 4  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12032        |
| 5  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5993         |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4827         |
| 7  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5490         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20860        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10552        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13960        |
| 11 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12028        |
| 12 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 16563        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137000 ms total latency. Provider mix: {'ollama\_remote': 12}

(full prompt+response transcripts available in `research/audit/275c21ac1aad.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/275c21ac1aad.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/275c21ac1aad.tar.gz` (if generated)